OFFICIAL PROMPTING PLAYBOOK FOR CLAUDE FABLE 5.2

Patterns for Effort Calibration, Persistence, Code Ownership, Subagents, and Clear Communication

Anthropic • Platform Documentation Adaptation • November 2026

Abstract. *Claude Fable 5.2 exhibits distinct prompting behaviors compared with earlier models. This playbook consolidates official guidance on effort levels, early-stopping mitigation, code ownership signals, progress updates, subagent expectations, caution accumulation, migration from prior models, and final-report clarity. Diagrams illustrate decision flows, effort trade-offs, and agent-team cycles. All prompt fragments are copy-ready. Replace bracketed fields as needed.*

1. Calibrate Effort

Effort is the primary control for thinking depth, latency, and token consumption. Start at **high** (the API default). Test changes against your own evaluations; do not carry over settings from Fable 5.1 or Opus 5.5.

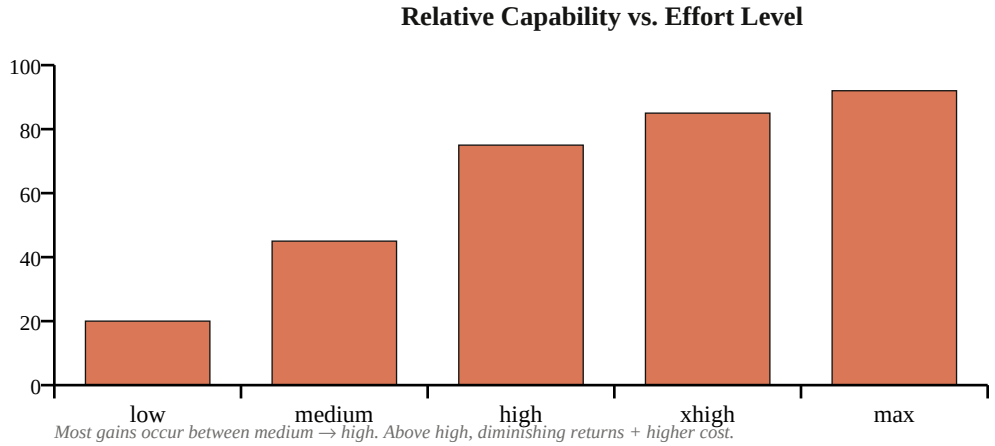


Figure 1. Relative capability by effort level (illustrative; most gains occur at the medium→high step).

low — quick back-and-forth, well-specified tasks, high-volume subagents.
medium — interactive coding, chat, everyday knowledge work.
high — hard coding, research, long agentic runs (recommended default).
xhigh — hardest long-running work where measured gains justify cost.
max — ceiling measurement or open-ended scientific/math problems.

At high and above the model thinks before almost every reply. Lower effort for trivial turns. Effort barely changes reply length; control length via prompt.

2. Finish the Whole Task

On long multi-step tasks (especially at low/medium effort) the model may end a turn early—listing remaining work or offering to continue. Add the following to the system prompt:

```
# Finishing work
Please avoid stopping while you still owe work the user asked for. Be persistent and do not stop
unless needed.
While status notes are welcome, as is clarifying with the user on open decisions, carry on with work
that does not depend on the user's answer without unnecessary stops.
If you notice yourself inviting the user to redirect you or offering to wait, instead proceed on the
next part of the task. You may meet ordinary obstacles like errors, timeouts, locked files, empty
results or failing tools. Diagnose them first, and when they are not real blockers, work through
them with the access you have (wait and retry, fix the request, use another tool or source) rather
than stopping unnecessarily.
If someone clearly decided something is a hard blocker, such as a file marked not to be touched,
access that was intentionally withheld, or a safety guardrail, leave it alone, say plainly what you
found, and look for another safe way to finish the task. Stopping before the task is complete is
very rarely merited...
```

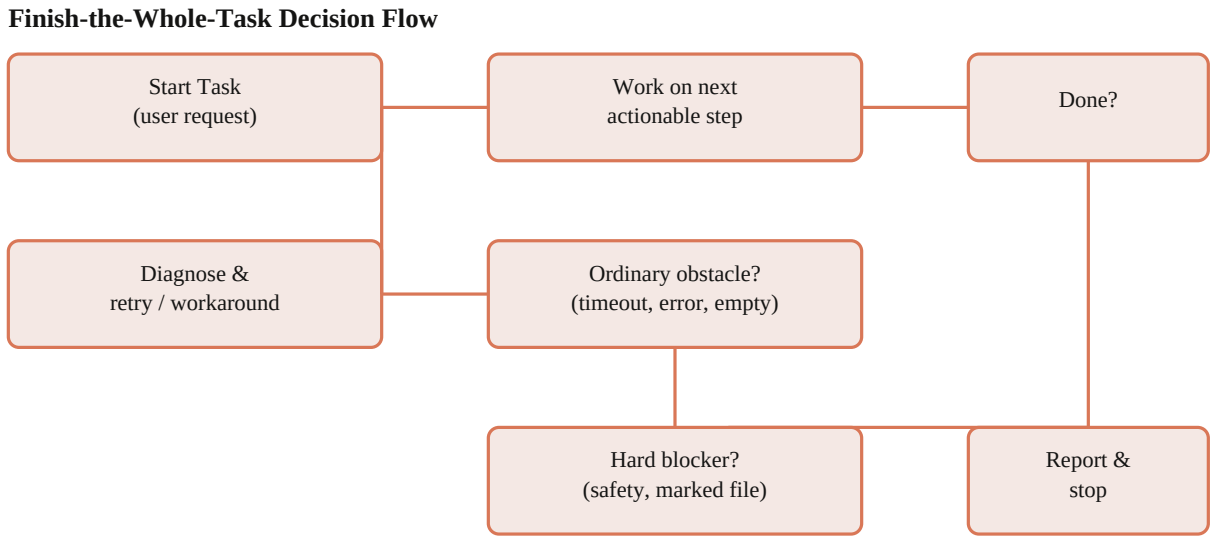


Figure 2. Decision flow distinguishing ordinary obstacles from hard blockers.

In Anthropic’s replay testing this text roughly halved sessions that ended with doable work remaining (+~30% output tokens). Still require explicit confirmation for risky or irreversible actions. When approving continuation, be specific (“Yes, run the migration on staging”).

3. Say Who Owns the Code

Ownership language strongly shapes behavior. When the prompt presents the codebase as the model’s own, it more often merges duplicated code and fixes nearby issues without being asked.

```
This codebase is yours: what it's for and how it's built are your call, and you'll keep building on
it.
```



Ownership language strongly influences structural improvements

Figure 3. Effect of ownership cue on structural changes (Anthropic small-task testing).

Leave the sentence out when the user owns the code and wants minimal change. A separate “no drive-by refactors” rule does not fully cancel the ownership effect.

4. Keep Users Informed During Long Turns

Long tool-calling runs can appear silent. Progress updates arrive as thinking blocks; set thinking.display to "updates" or "summarized" (beta). Recommended system-prompt addition:

```
Before you start, say in a line what you're about to do; brief updates while you work help the user follow along. Close with a short recap that stands on its own (what you found, what you did, and what's next), so a reader who only sees the last message has the full picture.
```

Harness option (after several consecutive silent tool steps): append a turn-scoped system message “The user hasn't heard from you in a while: say in a few words what you're doing, then continue.”

5. Set Expectations for Subagents

Delegation volume depends on the harness and tool descriptions. If the model starts more subagents than needed, revise the subagent-start tool description to state cost first, when delegation is worthwhile, and how to brief a subagent. In testing this reduced delegation ~33% on short tasks with unchanged success rate.

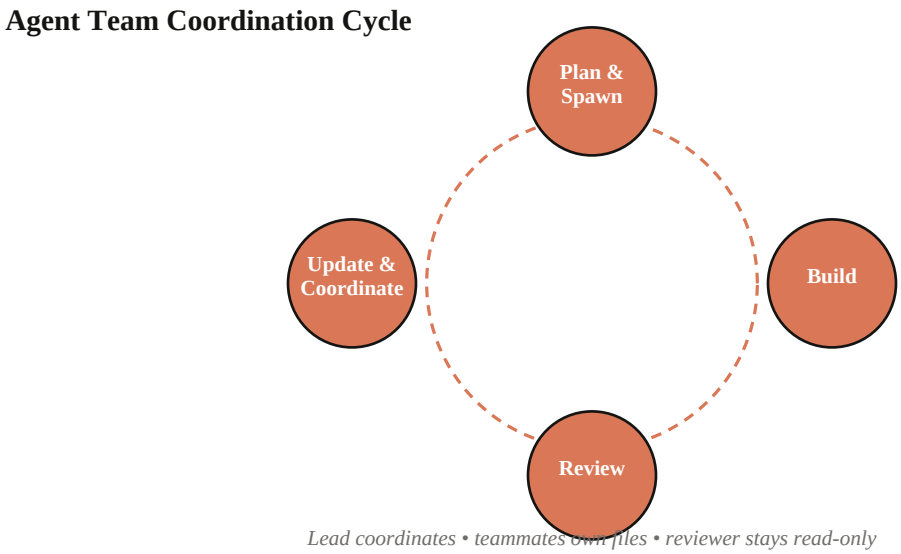


Figure 4. Recommended agent-team coordination cycle (inspired by coding-agent guidance).

At xhigh/max the model may use subagents for self-review—a behavior not observed on Fable 5.1. Check which model your harness runs subagents on by default.

6. Coding Agent Team Prompting Guide

For multi-agent coding work, structure the lead prompt with clear tagged blocks. The following is a copy-ready template (replace bracketed fields).

```
<title>
Fable 5.2 Coding Agent Team Prompting Guide
<purpose>
Build and lead a team of coding agents to ship [FEATURE] in [REPO].
It matters because [WHO NEEDS IT] will use it to [WHAT IT UNLOCKS].
Done means [ACCEPTANCE CRITERIA] and a passing run of [TEST COMMAND].
<team>
You are the lead. Create an agent team, not plain subagents.
Spawn 3–5 teammates. Add one only when the work is truly parallel.
Use an architect, [BACKEND] builder, [FRONTEND] builder, test engineer, reviewer.
Reuse role definitions from .claude/agents/ where they already exist.
Run builders on [BUILDER MODEL] and name that model in each spawn prompt.
Give every teammate its own files. No two teammates edit the same file.
Keep the reviewer read-only, outside the build, with a fresh context.
<process>
Read the repo first. Reduce the goal to one plan with clear module seams.
Split it into 5–6 tasks per teammate, each a self-contained deliverable.
Mark dependencies so blocked tasks wait and free tasks get claimed.

For every teammate write a spawn prompt with:
TEAMMATE [NAME] – [ROLE] – [MODEL]
Goal: ... Owns: ... Context: ... Done: ... Reports: ...

Coordinate; do not implement. Keep working while teammates run...
<delivery>
Return the team roster, file ownership map, and task list with dependencies.
Then run the team to the end: integrated code, passing tests, reviewed diff.
Do not stop at a plan, a task list, or a partial build.
Check every progress claim against real tool output before you report it.
If tests fail, say so and show the output. Shut down teammates when done.
Write the final report for a reader who saw none of the work...
<voice>
Act as a staff engineer, tech lead, and careful code reviewer.
Be decisive in planning, strict about evidence, and brief in updates.
Treat this codebase as [YOURS TO IMPROVE / NOT YOURS: CHANGE AS LITTLE AS POSSIBLE].
Do the simplest thing that works. No features or refactors beyond the task.
```

7. Keep Caution from Piling Up

Cautious notes, memory files, and inter-agent messages accumulate. Fable 5.2 follows them closely, sometimes asking for unnecessary approvals or holding back routine work. Remedies:

- Remove obsolete notes (e.g., “never retry”).
- Explicitly name steps that need no approval (retries for safe calls).
- Write exceptions into the rule itself rather than as separate lines.
- Check what reviewer agents attach to context.

8. Review Instructions Written for Earlier Models

Before migrating, re-examine standing delegation instructions, token-budget text in prompts, harness edits to earlier turns (breaks preserved thinking), “treat that answer as done” lines, instructions to think less, and overly strict style rules. Delete or rewrite cautions whose original reason no longer holds.

9. Make Reports Easy to Follow

Final reports and handoffs are read by people who have not seen the tool calls. Add:

```
# Communicating clearly
Write for an intelligent but low-context reader. The person reading has not seen your tool calls,
your reasoning or the files you opened...
Put the key points in the first few sentences. Practice progressive disclosure.
Use plain words and clear full sentences. Write mostly in the style of ASD-STE100 Simplified
Technical English: short sentences, one idea per sentence, active voice, and the same word for the
same thing.
Use a paragraph of flowing prose where ideas connect, and keep bullets for lists of separate items.
```

10. Safeguard Refusals

A declined request returns stop_reason: "refusal" with a category (cyber, bio, frontier_llm, reasoning_extraction, general_harms). Benign work can occasionally trigger classifiers; verification programs exist for legitimate cyber and life-sciences use. Remove any instructions that ask the model to write its reasoning into the response text.

11. Where to Start (Migration Checklist)

1. Set effort to high explicitly; run evals at medium and xhigh as well.
2. Review system prompts, skills, agent files, memory, and harness text. Remove or test the legacy instructions listed above.
3. Run your longest agentic tasks and note early stops, unnecessary approvals, silence, over-delegation, or unwanted structural changes.
4. Apply the corresponding guidance one change at a time and measure quality versus token cost.

This playbook adapts official Anthropic documentation for Claude Fable 5.2. Always verify against the latest platform docs and your own evaluations. Diagrams are illustrative.